1.0 Introduction

1.1 Literature Review

This project was both instructive and challenging in helping me deeply understand Hashset and Hashmap concepts in Java, improve my coding skills, and develop the ability to create my own Hashmap and Hashset classes (GTUHashSet & GTUHashMap) from scratch. I completed the project in Java as part of my CSE222 course. In addition to Hashmap and HashSet, I created my own generic ArrayList class called MyList and implemented my own Iterator class to easily traverse through the hashset.

To enhance efficiency, I integrated various methods such as pruning and next prime number calculations into the project. I also used linear probing to prevent collisions that might occur within the hash table.

1.2 Objectives

- Implementing Hashmap and Hashset classes from scractch.
- Avoid collision in hashmap by using linear probing methods.
- Rehash to a larger capacity by using next prime capacity method.
- Using tombstone method to deleting item from hash table.
- Implementing my own generic ArrayList class from scracht called MyList.
- Implementing my own Iterator class from scracth.
- Using pruning method to increase efficiency.

2.0 Methodology

GtuHashMap Class

Firstly, I implemented a fully functional hash map from scratch which is named GTUHashMap. Thanks to this class, I keep key-value pairs. While adding elements to the hash table, if capacity is full, then I increase the size of the hash table using the next prime number method. In this GTUHashMap class, I used open addressing with linear probing. Thanks to this, I avoid collisions. This class supports these methods:

- **void put(K key, V value)** = Inserts a key-value pair into the hash table.
- **V get(K key)** = Returns the value with the specified key.
- **void remove(K key)** = Removes the mapping for the specified key from this map if present. Uses tombstone method for removing.
- **boolean containsKey(K key)** = Returns true if this map contains a mapping for the specified key.
- **int size()** = Returns the number of key-value mappings in this map.
- **Object[] getKeys()** = Returns an array of all the keys in this map.
- **void resize()** = Resizes the hash table with the next prime number method.
- **int getCollisionCount()** = Function to get collision count.
- **void resetCollisionCount()** = Resets collisionCount.

GTUHashSet

I built a set that internally uses my GTUHashMap to store elements. In this class, I implemented an inner class named GTUHashSetIterator. Thanks to this class, I implemented the Iterator interface from scratch to traverse the set easily. This set supports these methods:

- **void add(E element)** = Adds the specified element to this set if it is not already present.
- **void remove(E element)** = Removes the specified element from this set if it is present.
- **boolean contains(E element)** = Returns true if this set contains the specified element.
- **int size()** = Returns the number of elements in this set.

SpellChecker

In this section, I take the words from the given Dictionary.txt file. Then I added the all words to GTUHashSet. I got help from the prunning method when adding words. I stored all the words from the Dictionary.txt file in different GTUHashSet objects based on their word lengths. So, I don’t search the entire dictionary when browsing to find possible suggestions. I check the sets with lengths between 2 less and 2 more than the input's letter count.

How to SpellChecker Works?

Firstly, I receive an input from the user then I check if the Dictionary contains this input. If the dictionary contains this input then I print correct on the terminal. If the input is not in the dictionary I call the **generateAllSuggestions** method and I send the input into this method as a parameter. This **generateAllSuggestions** method returns a GTUHashSet which contains all the suggestions generated.

After these operations I navigate this set one by one with help of the Iterator that I implemented from the scratch. In this navigation I check for the dictionary contains these suggestions or not. If any dictionary contains any suggestions, I put this suggestion into validSuggestions set. And at the final I print all the valid suggestions on the terminal. At the end of the this process I print these informations on the terminal:

- Count of all suggestions

- Count of valid suggestions

- Count of collisions

And this function call these functions:

- BufferedReader and FileReader = Required to give the words from Dictionary.txt

- **void resetCollisionCount()** = Resets collision counts

- **GTUHashSet<String> generateAllSuggestions(String input)** = Generates all possible suggestions for a given input.

- **long System.nanoTime()** = Returns the current value of the running JVM’s high-resolution time source, in nanoseconds.

MyList

MyList is a generic resizable array implementation. It provides methods for adding annd getting elements, and checking the size. MyList class support these methods:

- **void reSize()** = Increase the capacity of the list

- **void add(E item)** = Adding an element to the list

- **void addList(MyList<E> list)** = Adding an other list to the this list

- **E get(int index)** = Returns the element at the specified position in this list.

- **boolean isEmpty()** = Returns true if this list contains no elements.

- **int size()** = Returns the number of elements in this list.

Operations Class

Operations class contains functions to generetate suggestions by applying various operations.I designed this class to perform a maximum of 10 thousand transactions to increase efficiency. This class supports these methods:

-String replaceAt(String str, int index, char newChar) = Replaces a character at a specific index with a another char.

-String insertCharToString(String input, int index, char newChar) = Inserts a character at a specific index in a string.

- boolean reachedLimit() = Checks if the suggestion limit has been reached.

-void resetOperationCount() = Resets operation count

-MyList<String> delete_1_char(String input) = Generates suggestions by deleting one character from the input string. And return MyList object which contains suggestions.

-MyList<String> delete_2_char(String input) = Generates suggestions by deleting two characters from the input string. And return MyList object which contains suggestions

- MyList<String> replace_1_char(String input) = Generates suggestions by replacing one character in the input string. And return MyList object which contains suggestions

-MyList<String> replace_2_char(String input) = Generates suggestions by replacing two characters in the input string.And return MyList object which contains suggestions.

-MyList<String> insert_1_char(String input) = Generates suggestions by inserting one character into the input string.And return MyList object which contains suggestions.

-void insert_2_char(String input) = Generates suggestions by inserting two characters into the input string. And adds suggestions into allSuggestions.

-void replace_and_insert(String input) = Generates suggestions by first replacing and then inserting a character.And adds suggestions into allSuggestions.

-void remove_and_replace(String input) = Generates suggestions by first removing and then replacing a character. And adds suggestions into allSuggestions.

-void allOperations(String input) = Applies all operations to generate suggestions for the input string.

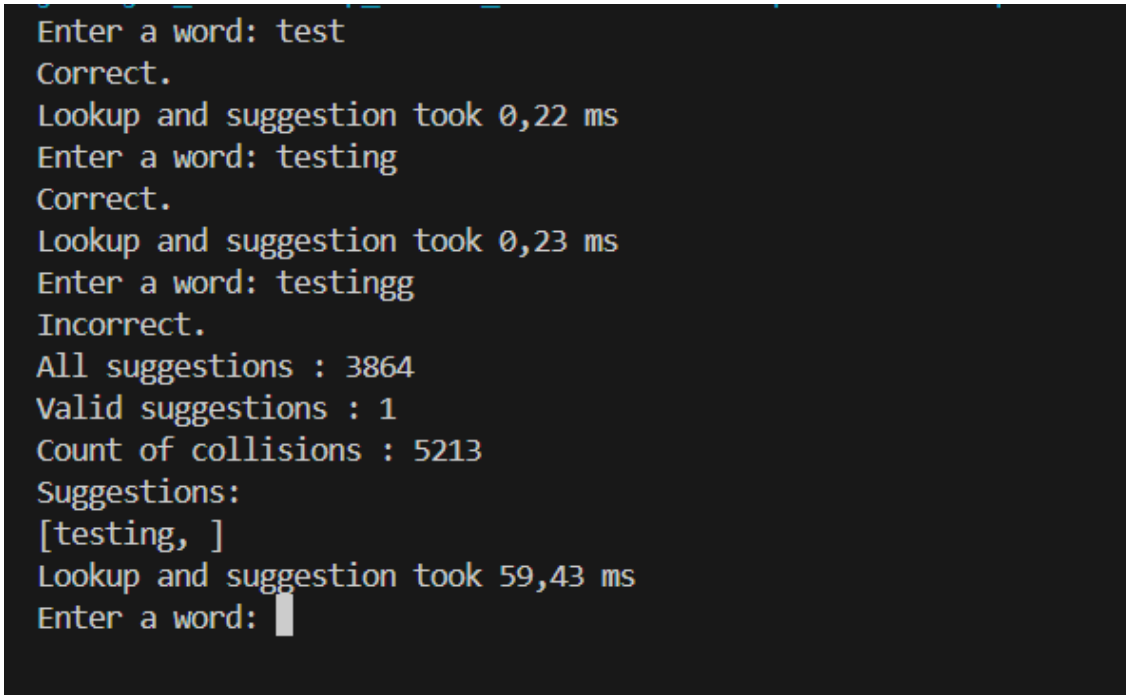
-GTUHashSet<String> getAllSuggestions = Generates all suggestions for the input string and returns them as a set.

3.0 RESULTS AND DISCUSSION

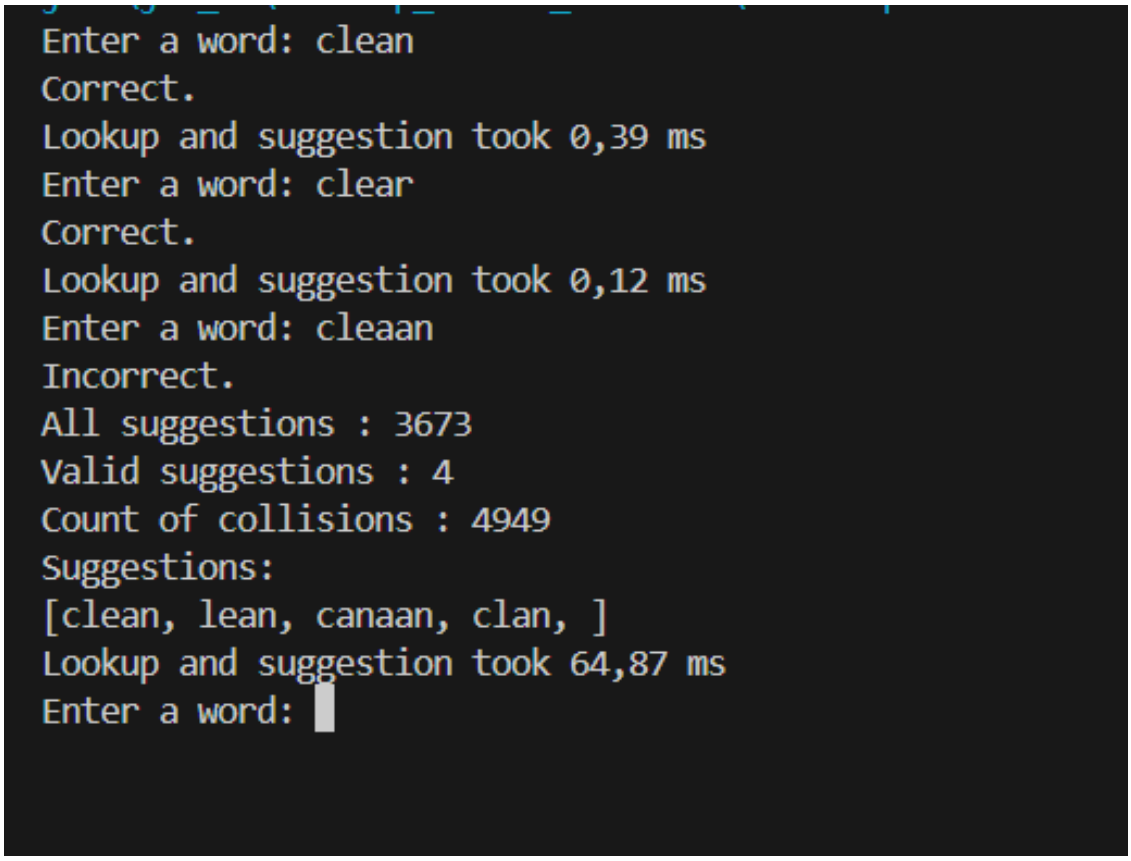
First of all, I want to say that the assignment was very instructive and challenging for me. I couldn't fully complete everything specified in the assignment PDF. For example, I used linear probing instead of quadratic probing because I couldn't fully understand the logic of quadratic probing. While generating suggestions, I initially didn't set a 10,000 operation limit, which caused the project's execution time to exceed 100ms. However, after implementing the 10,000 operation limit, the assignment's execution time dropped to below 50ms. I can summarize the 10 thousand operations limit as follows: While generating suggestions based on the length of the word, I named each change I made on each letter as an operation. And in total, after performing 10,000 operations, I prevented it from performing any additional operations. Thus, the program run much faster, but it caused a problem: Since suggestion generation stops after 10,000 operations, I can’t get all the possible suggestions that could be generate. This prevents the program from working 100% correctly, especially with longer words. The two functions that increased the program's execution time the most were insert_2_char and replace_2_char because they contain three nested for loops. I did my best but unfortunately I couldn't improve their optimization. Also when I completed the code part, there was no error handling in spellChecker for inputs with 1 or 2 characters. However, during testing, I observed that these inputs were terminating the program, so I added error handling for inputs with 1 or 2 characters as well. (The test is in the output section.)

On the other hand I tried to write clean code and establish good OOP design in the assignment as much as possible. As required in the assignment, I created everything from scratch myself, including the list and iterator classes, which helped me refresh my previous knowledge. Below are some outputs from the assignment:

OUTPUT - 1



OUTPUT – 2



OUTPUT – 3

```
Enter a word: gorkem
Incorrect.
All suggestions : 3678
Valid suggestions : 9
Count of collisions : 5001
Suggestions:
[gore, worker, mortem, corker, worked, corked, yorker, porker, forked, ]
Lookup and suggestion took 68,87 ms
Enter a word: gork
Incorrect.
All suggestions : 2931
Valid suggestions : 103
Count of collisions : 5108
Suggestions:
[dark, mark, ok, port, gory, yolk, cord, core, nook, rock, or, cork, lord, corm, corn, lore, lori, souk, turk, hora, cory, germ, perk, horn, b
onk, hors, nark, dora, jock, more, sock, dorm, morn, mors, lurk, sark, mort, dory, book, kook, took, conk, gawk, nora, gook, word, wore, norm,
 work, worm, bark, honk, worn, murk, kirk, sorb, cook, look, pock, sore, garb, gird, monk, park, sort, girl, ford, gars, hook, lock, cock, gir
o, fore, gary, girt, fori, fork, form, go, lark, fort, folk, bord, bore, hock, tore, jerk, born, torn, hark, tors, tort, dock, tory, pore, roo
k, yore, mock, gore, soak, pork, york, porn, ]
Lookup and suggestion took 26,40 ms
Enter a word:
```

OUTPUT – 4

```
Enter a word: blabla
Incorrect.
All suggestions : 3677
Valid suggestions : 3
Count of collisions : 4893
Suggestions:
[baba, blab, bala, ]
Lookup and suggestion took 69,86 ms
Enter a word: aaaaaaaaaaaaaaaaa
Incorrect.
All suggestions : 3421
Valid suggestions : 0
Count of collisions : 4380
Suggestions:
[]
Lookup and suggestion took 7,91 ms
Enter a word: gets
Correct.
Lookup and suggestion took 0,32 ms
Enter a word: getz
Incorrect.
All suggestions : 2932
Valid suggestions : 29
Count of collisions : 4906
Suggestions:
[goth, lets, ritz, et, pete, beta, get, beth, jets, wets, pets, veto, bets, vets, gits, mete, seta, fete, meth, gate, peez, nets, seth, gets,
benz, mets, yeti, sets, gatt, ]
Lookup and suggestion took 28,03 ms
Enter a word:
```

OUTPUT – 5

```
Enter a word: kk
Incorrect.
All suggestions : 729
Valid suggestions : 175
Count of collisions : 719
Suggestions:
[us, ut, ok, bo, om, on, id, op, ie, or, os, if, ii, bt, ow, ox, il, in, oz, vc, vi, is, it, pa, iv, pc, ix, pe, by, ca, ph, pi, vu, cb, cc, p
m, cd, po, cy, ce, ch, ps, cm, co, cs, wc, we, jo, de, jp, jr, dl, qc, wo, do, dr, du, ka, kb, kc, ql, kg, qt, km, ed, ee, ko, eg, eh, kr, ku,
 ra, el, em, en, eo, rd, re, er, es, et, la, xi, lc, ex, ey, le, xu, xv, li, xx, k, fa, lm, lo, fe, lp, fi, sc, yo, se, fo, si, fs, ft, fu, ma
, mb, md, fy, me, mg, so, mi, sr, mk, ml, gb, gc, mm, mo, mp, mr, ms, gi, mu, st, sw, gm, mx, go, my, ta, tb, te, th, ti, aa, ab, to, ne, ad,
ae, ah, ai, al, tv, ha, tx, am, no, he, an, ao, as, hi, nt, at, nw, uc, au, ho, ay, ba, bc, be, uk, bi, um, un, od, oe, of, up, oh, ]
Lookup and suggestion took 39,05 ms
Enter a word: 5
Incorrect.
All suggestions : 729
Valid suggestions : 26
Count of collisions : 609
Suggestions:
[a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z, ]
Lookup and suggestion took 3,32 ms
Enter a word:
```

OUTPUT – 6

```
Enter a word: 1234
Incorrect.
All suggestions : 3177
Valid suggestions : 0
Count of collisions : 4956
Suggestions:
[]
Lookup and suggestion took 51,76 ms
Enter a word: 12
Incorrect.
All suggestions : 783
Valid suggestions : 174
Count of collisions : 1800
Suggestions:
[us, ut, ok, bo, om, on, id, op, ie, or, os, if, ii, bt, ow, ox, il, oz, in, vc, vi, is, it, pa, iv, pc, ix, pe, by, ca, ph, pi, vu, cb, cc, p
m, cd, po, cy, ce, ch, ps, cm, co, cs, wc, we, jo, de, jp, jr, dl, qc, wo, do, dr, du, ka, kb, kc, ql, kg, qt, km, ed, ee, ko, eg, eh, kr, ku,
 ra, el, em, en, eo, rd, re, er, es, et, la, xi, lc, ex, ey, le, xu, xv, li, xx, fa, lm, lo, fe, lp, fi, sc, yo, se, fo, si, fs, ft, fu, ma, m
b, md, fy, me, mg, so, mi, sr, mk, ml, gb, gc, mm, mo, mp, mr, ms, gi, mu, st, sw, gm, mx, go, my, ta, tb, te, th, ti, aa, ab, to, ne, ad, ae,
 ah, ai, al, tv, ha, tx, am, no, he, an, ao, as, hi, nt, at, nw, uc, au, ho, ay, ba, bc, be, uk, bi, um, un, od, oe, of, up, oh, ]
Lookup and suggestion took 14,99 ms
Enter a word:
```

OUTPUT – 7

```
Enter a word: /ga*za
Incorrect.
All suggestions : 3834
Valid suggestions : 1
Count of collisions : 4895
Suggestions:
[gaza, ]
Lookup and suggestion took 64,56 ms
Enter a word: -st-za
Incorrect.
All suggestions : 3834
Valid suggestions : 0
Count of collisions : 5189
Suggestions:
[]
Lookup and suggestion took 37,23 ms
Enter a word:
```

OUTPUT – 8

```
Enter a word: dummyinputword
Incorrect.
All suggestions : 3782
Valid suggestions : 0
Count of collisions : 4709
Suggestions:
[]
Lookup and suggestion took 37,28 ms
Enter a word: worzaroaz
Incorrect.
All suggestions : 4071
Valid suggestions : 0
Count of collisions : 4896
Suggestions:
[]
Lookup and suggestion took 37,50 ms
Enter a word: slzag
Incorrect.
All suggestions : 3391
Valid suggestions : 3
Count of collisions : 5536
Suggestions:
[sag, lag, slag, ]
Lookup and suggestion took 28,84 ms
Enter a word:
```

4.0 CONCLUSIONS/ RECOMMENDED FUTURE WORK

In this project, I developed a program that checks whether the word received from the user exists in the dictionary, and if it is not in the dictionary, suggests words that are close to the entered input.I significantly improved my coding and algorithm development skills by creating Hashset and Hashmap structures also developing List and Iterator structures from scratch by myself. I learned new methods such as pruning technique, tombstone method, and probing. The most challenging aspect of the assignment for me was establishing correct and efficient algorithms while generating suggestions. I think I especially need to improve the functions that contain three nested for loops. As a result, I managed to develop an application that provides basic functionality and operates within acceptable time frames.

Recommended Features and Improvements:

1- Algorithm Optimization: As I mentioned above, the methods I used while generating suggestions can be redesigned with more efficient algorithms.
2- Quadratic Probing Implementation: Hash collisions can be managed more effectively by using quadratic probing instead of the current linear probing.
3- Memory Usage Optimization: The memory usage of data structures can be made more efficient.

These improvements can significantly enhance both the performance and user experience of the application.

REFERENCES

1- Oracle. (2023). Java Documentation - Collections Framework. Retrieved from https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/package-summary.html

2- Slides we use in the lessons
3- The book we use in class: Data Structures: Abstraction and Design Using Java 2nd Edition